

```

8 --w-----w- 1 student student 0 Sep 19 21:47 road-trip.png
9 student@uso:~$ chmod u=rw,g=r,o= road-trip.png
10 student@uso:~$ ls -l road-trip.png
11 -rw-r----- 1 student student 0 Sep 19 21:47 road-trip.png
12 student@uso:~$ chmod 644 road-trip.png
13 student@uso:~$ ls -l road-trip.png
14 -rw-r--r-- 1 student student 0 Sep 19 21:47 road-trip.png

```

Listing 5.20: Modificarea permisiunilor unui fișier

Forma în octal a permisiunilor la **chmod** este preferată atunci când vrem să configurăm un nou set de permisiuni unui fișier. Când vrem să actualizăm permisiunile și modificările sunt mici, este de preferat să folosim forma în octal a permisiunilor.

5.5.3.1 Permisuni de creare pentru fișiere

Atunci când creăm un fișier sau director, folosind, de exemplu, comenzile **touch** și **mkdir**, le sunt asociate metadate implicite la creare. Utilizatorul și grupul fișierului vor fi utilizatorul și grupul procesului, care sunt, de obicei, utilizatorul și grupul primar al utilizatorului care a lansat comanda. Stabilirea permisiunilor la creare ține de **masca de creare a shellului** (numită și **umask**).

Masca de creare a shellului este o valoare pe biți care este folosită astfel:

- Pentru fișiere se face un **SI logic** între valoarea 666 (octal) și masca inversată.
- Pentru directoare se face un **SI logic** între valoarea 777 (octal) și masca inversată.

Deosebirea între fișiere și directoare apare pentru că directoarele au nevoie de permisiuni de execuție pentru toate operațiile. Iar fișierele nu au nevoie de permisiuni de execuție; dacă este cazul acestea trebuie acordate explicit.

Exemple sunt prezente în Tabelul 5.6.

Mască de creare	Mască inversată	Permisuni de creare fișier	Permisuni de creare director
022	755	644	755
002	775	664	775
077	700	600	700

Tabelul 5.6: Permisuni de creare fișier în funcție de mască

Afișarea și modificarea măștii shellului se face folosind comanda **umask**. Fără parametru această comandă afișează masca de creare, iar cu parametru modifică masca de creare, ca în Listing 5.21.

```

1 student@uso:~$ umask
2 0002
3 student@uso:~$ touch ana.txt
4 student@uso:~$ ls -l ana.txt
5 -rw-rw-r-- 1 student student 0 Sep 19 21:50 ana.txt
6 student@uso:~$ mkdir bogdan.dir
7 student@uso:~$ ls -ld bogdan.dir

```

```

8 drwxrwxr-x 2 student student 4096 Sep 19 21:50 bogdan.dir
9 student@uso:~$ umask 0022
10 student@uso:~$ umask
11 0022
12 student@uso:~$ touch corina.txt
13 student@uso:~$ ls -l corina.txt
14 -rw-r--r-- 1 student student 0 Sep 19 21:51 corina.txt
15 student@uso:~$ mkdir dan.dir
16 student@uso:~$ ls -ld dan.dir
17 drwxr-xr-x 2 student student 4096 Sep 19 21:51 dan.dir

```

Listing 5.21: Masca de creare a fișierelor (umask)

Tot în Listing 5.21 (liniile 9–17) sunt exemple de modificare a umask-ului și crearea de fișiere și directoare pentru a vedea cum modificarea valorii umask duce la modificarea măștii de creare a shellului și, astfel, a permisiunilor de creare pentru fișiere și directoare.

5.5.4 Biți speciali de acces: setuid, setgid, sticky

Atunci când afișăm masca de creare a shellului, observăm că sunt patru cifre octale afișate. În mod normal ar trebui să fie trei cifre afișate, cu permisiuni pentru `user`, `group` și `others`. Prima cifră afișată reprezintă un set special de trei de biți numiți `setuid`, `setgid` și `sticky`, și sunt activați în această ordine. Astfel, pentru permisiunile de mai jos avem explicațiile:

- 4755: este activat bitul `setuid`, utilizatorul are permisiuni complete, grupul are permisiuni de citire și execuție, ceilalți au permisiuni de citire și execuție
- 2755: este activat bitul `setgid`, celelalte permisiuni sunt ca mai sus
- 1755: este activat bitul `sticky`, celelalte permisiuni rămân la fel

Bitul `setuid` are sens când este folosit pe fișiere executabile. Adaugă funcționalitatea de set user ID on execution. Când un fișier executabil are activat bitul `setuid`, procesul creat din acest fișier executabil va avea utilizatorul ale acestui fișier (de obicei `root`) nu al utilizatorului care a rulat comanda. Mai concret, dacă rulăm ca utilizator `student` executabilul `myexec` care are activat bitul `setuid` și are ca utilizator `root`, atunci procesul creat din executabilul `myexec` va rula ca utilizatorul `root`.

Bitul `setuid`, prezentat și în Secțiunea 5.2.3 este folosit atunci când anumite utilitare necesită privilegii pentru accesarea unor fișiere speciale atunci când rulează. În Listing 5.22, executabilul `/usr/bin/passwd` are activat bitul `setuid` pentru că procesul creat va trebui să acceseze fișierul `/etc/shadow`, fișier accesibil doar utilizatorilor privilegiați.

```

1 student@uso:~$ ls -l /usr/bin/passwd
2 -rwsr-xr-x 1 root root 59640 Jan 25 2018 /usr/bin/passwd
3 student@uso:~$ stat /usr/bin/passwd
4  File: /usr/bin/passwd
5  Size: 59640          Blocks: 120          IO Block: 4096   regular
   file
6 Device: 801h/2049d    Inode: 656195      Links: 1
7 Access: (4755/-rwsr-xr-x)  Uid: (  0/   root)   Gid: (  0/   root)
8 Access: 2018-08-21 11:41:23.741306286 +0300
9 Modify: 2018-01-25 17:09:22.000000000 +0200
10 Change: 2018-08-06 17:34:29.848000000 +0300

```

11 Birth: -

Listing 5.22: Folosirea bitului setuid

Vedem în rezultatul rulării comenzilor prezența cifrei 4 pe prima poziție în forma octală a permisiunilor și a caracterului `s` în forma literală a permisiunilor în locul caracterului de execuție pentru utilizator. Aceasta înseamnă prezența bitului `setuid` în permisiunile fișierului.

Bitul `setgid` are sens când este folosit pe executabile și pe directoare. Adaugă funcționalitatea de *set group ID on execution*. Pe executabile are același efect ca bitul `setuid` doar că pentru grup.

Când un director are activat bitul `setgid`, viitoarele subdirectoare și fișiere ale acestuia vor avea ca grup în metadata grupul directorului, nu grupul utilizatorului care a rulat comanda. Viitoarele subdirectoare vor moșteni bitul `setgid` astfel că ce se creează nou în ierarhie va avea mereu același grup. Este util pentru a crea un spațiu partajat în care diferiți utilizatori să creeze fișiere dar să rămână accesibile între ei câtă vreme fac parte din același grup. Mai concret, dacă directorul `top/` are grupul `heroes` și are activat bitul `setgid`, viitoarele fișiere și subdirectoare vor avea tot grupul `heroes`, indiferent ce utilizatori au creat intrările.

Listing 5.23 prezintă cum activarea bitului `setgid` pe un director duce la crearea de intrări care moștenesc grupul directorului.

```
1 student@uso:~$ ls -ld top/
2 drwxr-xr-x 2 student heroes 4096 Sep 19 21:54 top/
3 student@uso:~$ sudo chmod g+s top/
4 student@uso:~$ ls -ld top/
5 drwxr-sr-x 2 student heroes 4096 Sep 19 21:54 top/
6 student@uso:~$ mkdir top/alarmar
7 student@uso:~$ ls -ld top/alarmar
8 drwxr-sr-x 2 student heroes 4096 Sep 19 21:55 top/alarmar
```

Listing 5.23: Folosirea bitului setgid

Bitul `sticky` are sens când este folosit pe directoare la care au acces de scriere mai mulți utilizatori, de exemplu directorul `/tmp`. Adaugă funcționalitatea de *restricted deletion flag*. Acest lucru înseamnă că un utilizator nu poate șterge fișierele altui utilizator, deși are permisiuni de scriere pe directorul care conține acele fișiere.

Directorul `/tmp` este cel care are activat bitul `sticky` pentru că oferă permisiuni complete tuturor utilizatorilor. Deși un utilizator are permisiuni complete pe acel director, nu poate șterge fișiere care nu-i aparțin, așa cum avem prezentat în Listing 5.24.

```
1 student@uso:~$ ls -ld /tmp/
2 drwxrwxrwt 13 root root 4096 Sep 19 21:54 /tmp/
```

Listing 5.24: Folosirea bitului sticky

5.6 Anexă: Resetarea parolei în Linux

Se poate întâmpla să fie uitată parola administrativă pe un sistem care rulează Linux. În această situație este nevoie de resetarea parolei contului `root`. Pentru aceasta avem două opțiuni: